



UNIVERSIDADE
FEDERAL DO CEARÁ
CAMPUS QUIXADÁ

Aula 6

Diagrama de Sequência

Disciplina: ANÁLISE E PROJETO DE SISTEMAS

Prof. Emanuel Coutinho

Agenda

- Conceitos
- Componentes
- Exemplos
- Exercícios



Como modelar o comportamento e a dinâmica de um sistema ?

Introdução

- Objetivo dos modelos vistos até agora é fornecer um entendimento do problema correspondente ao sistema a ser desenvolvido
 - Esses modelos deixam algumas perguntas sem respostas
- No modelo de casos de uso:
 - Quais são as operações que devem ser executadas internamente ao sistema?
 - A que classes estas operações pertencem?
 - Quais objetos participam da realização deste caso de uso?

Introdução

- No modelo de classes de análise:

De que forma os objetos colaboram para que um determinado caso de uso seja realizado?

Em que ordem as mensagens são enviadas durante esta realização?

Que informações precisam ser enviadas em uma mensagem de um objeto a outro?

Será que há responsabilidades ou mesmo classes que ainda não foram identificadas?

Introdução

- Resposta:
 - Modelo de interações
- Representa **mensagens** trocadas entre objetos para a execução de cenários dos casos de uso do sistema
- A construção dos diagramas de interação é uma consolidação do entendimento dos **aspectos dinâmicos** do sistema



Introdução

Diagramas de interação representam como o sistema age internamente para que um ator atinja seu objetivo na realização de um caso de uso

A modelagem de um sistema OO normalmente contém diversos diagramas de interação

O conjunto de todos os diagramas de interação de um sistema constitui o seu ***modelo de interações***

Introdução

- Objetivos da construção do modelo de interação:
 - Obter informações adicionais para completar e aprimorar outros modelos (principalmente o modelo de classes)
 - Quais as operações de uma classe?
 - Quais os objetos participantes da realização de um caso de uso (ou cenário deste)?
 - Para cada operação, qual a sua assinatura?
 - Uma classe precisa de mais atributos?
 - Fornecer aos programadores uma visão detalhada dos objetos e mensagens envolvidos na realização dos casos de uso

Tipos de diagrama de interação

- Três tipos de diagrama de interação na UML 2.0:
 - Diagrama de sequência, diagrama de comunicação e diagrama de visão geral da interação
- Diagrama de sequência e de comunicação são equivalentes

Diagrama de sequência: foco nas mensagens enviadas no decorrer do tempo

Diagrama de comunicação: foco nas mensagens enviadas entre objetos que estão relacionados

Diagrama de visão geral de interação. Pode ser utilizado para apresentar uma visão geral de diversas interações entre objetos, cada uma delas representada por um diagrama de interação. Diagrama é útil para ***modularizar*** a construção do diagramas de sequência (ou de comunicação)

Diagrama de Sequência

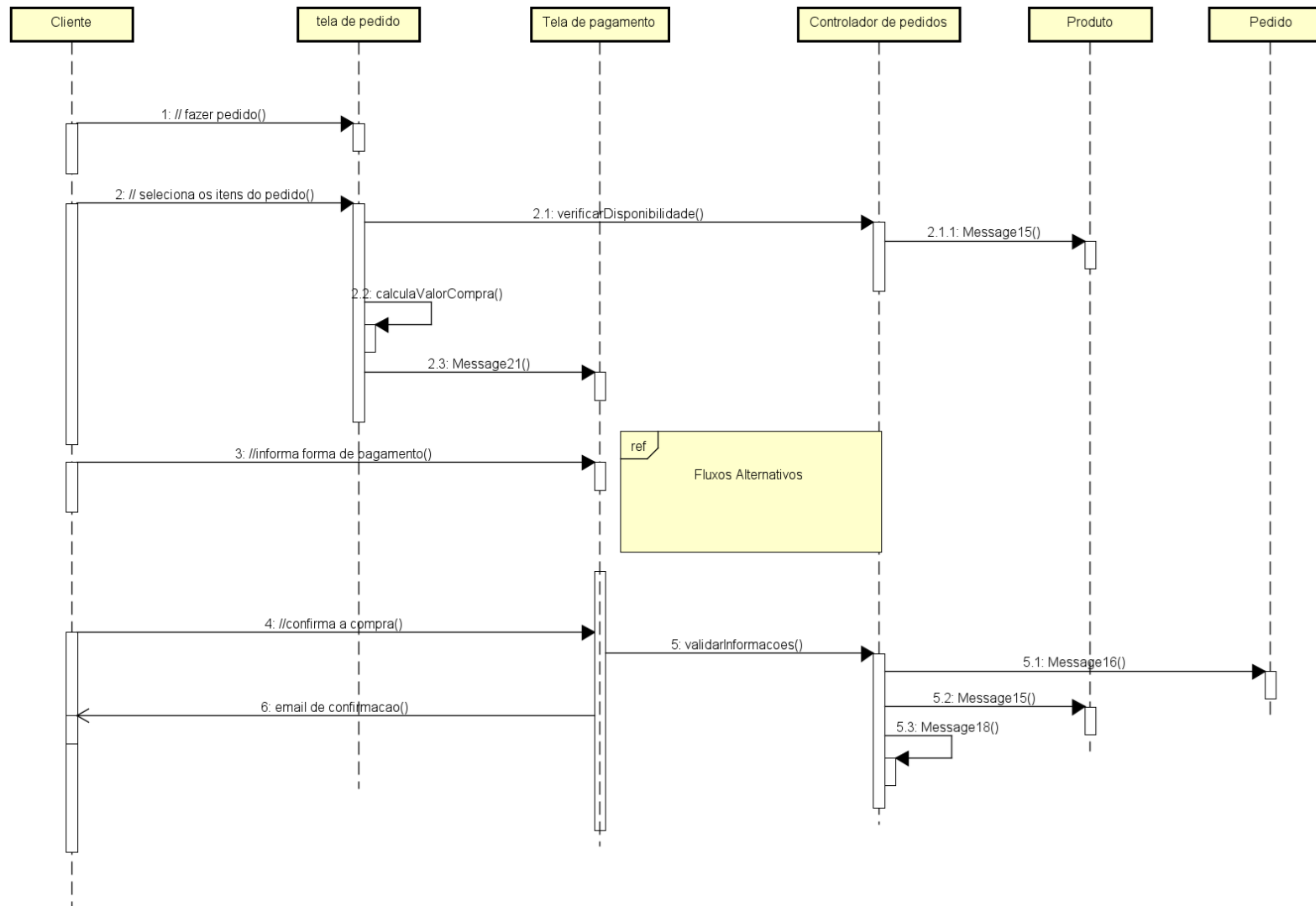


Diagrama de Comunicação

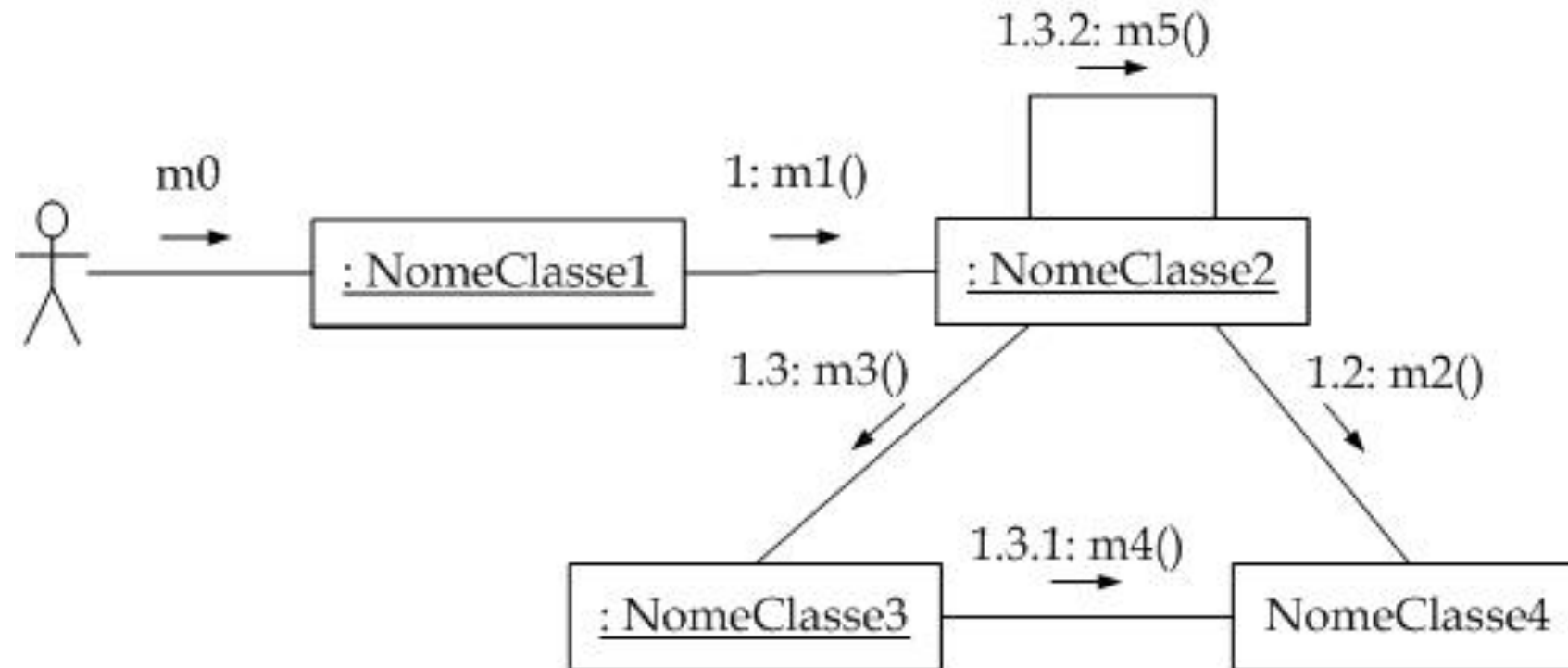


Diagrama de Sequência

Diagrama de sequência

- Os objetos participantes da interação são organizados na **horizontal**
- Abaixo de cada objeto existe uma linha (**linha de vida**)
- Cada linha de vida possui o seu **foco de controle**
 - Quando o objeto está fazendo algo

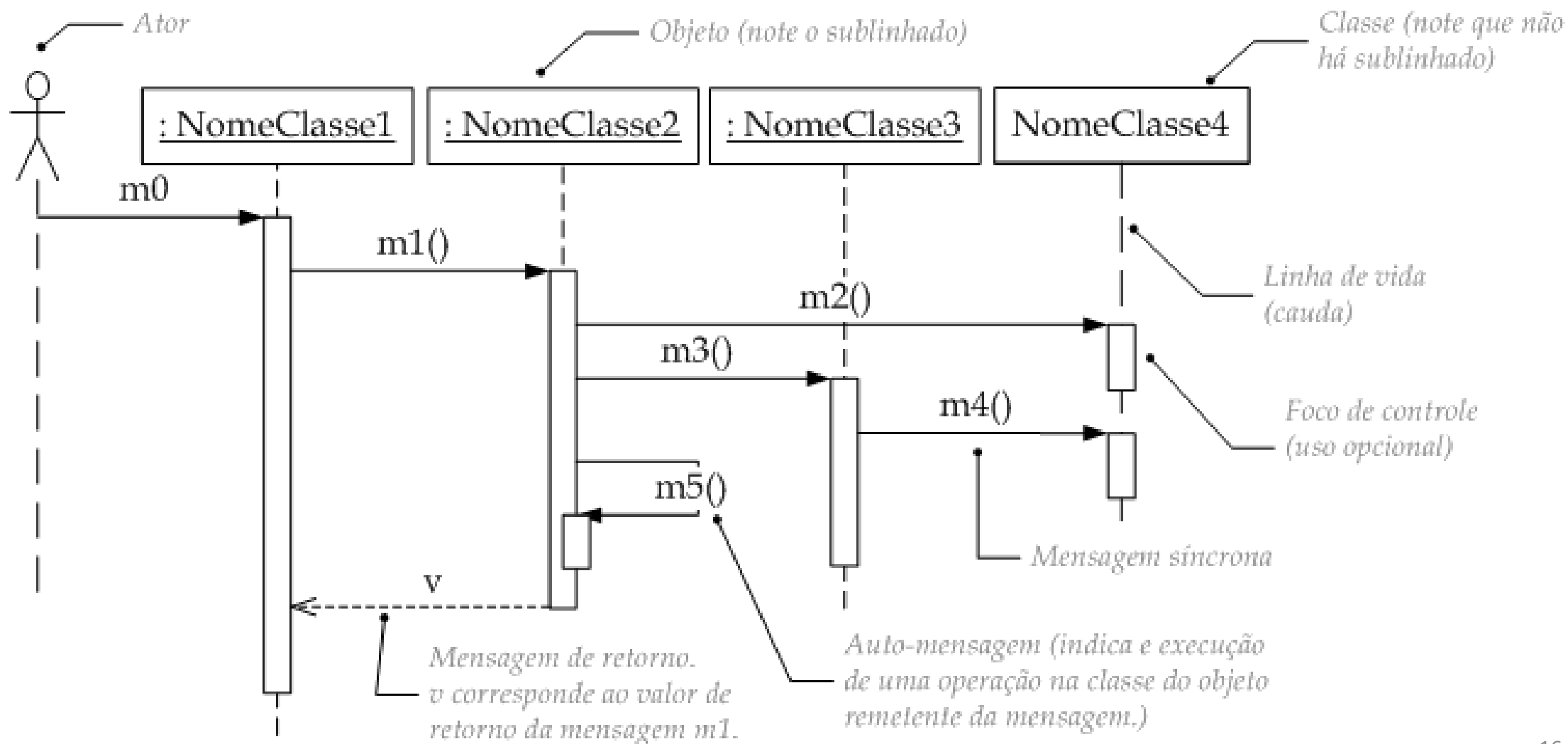
Diagrama de sequência

- Mensagens entre objetos são representadas com **linhas horizontais** rotuladas partindo da linha de vida do objeto remetente e chegando a linha de vida do objeto receptor
- Posição vertical das mensagens permite deduzir a **ordem** na qual elas são enviadas
- Ordem de envio de mensagens em um diagrama de sequência pode ser deduzida a partir das **expressões de sequência**
- **Criação e destruição** de objetos podem ser representadas

Elementos gráficos de um DS

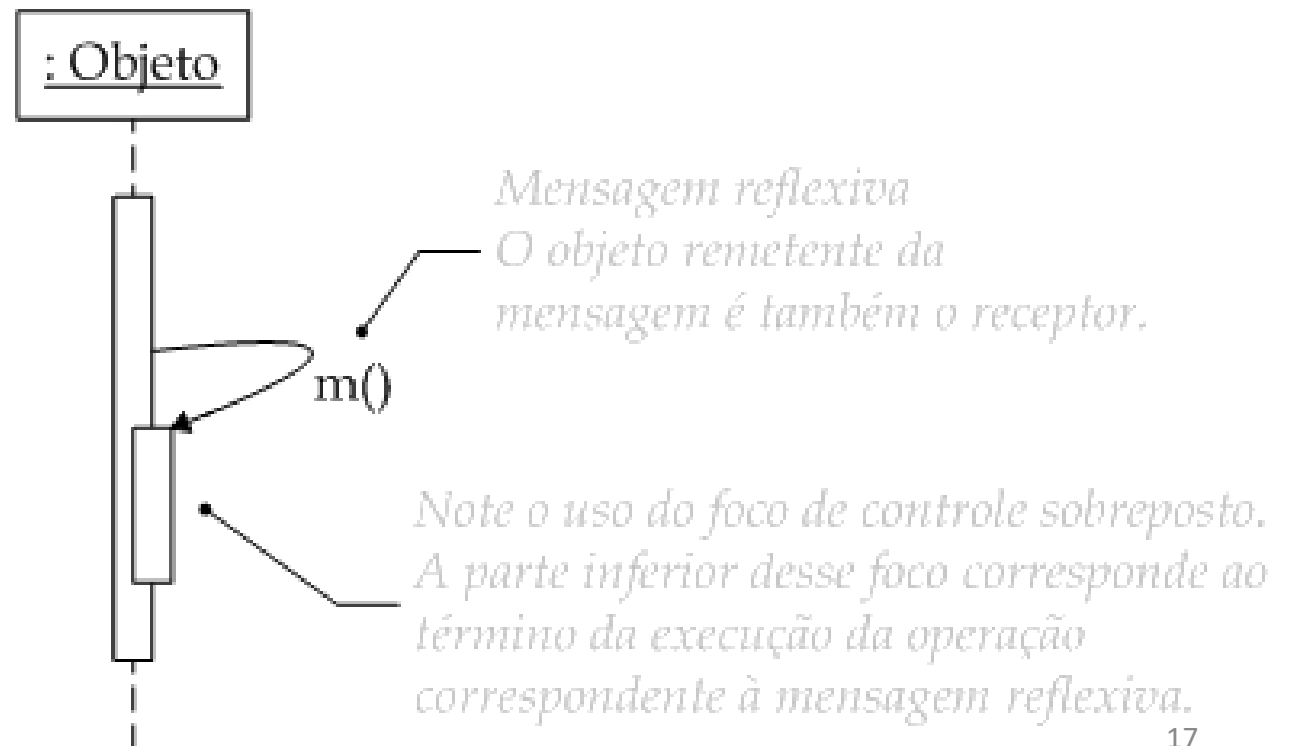
- Elementos básicos em um diagrama de sequência:
 - Atores
 - Objetos, multiobjetos e classes
 - Mensagens
 - Linhas de vida e focos de controle
 - Criação e destruição de objetos
 - Iterações

Elementos gráficos de um DS

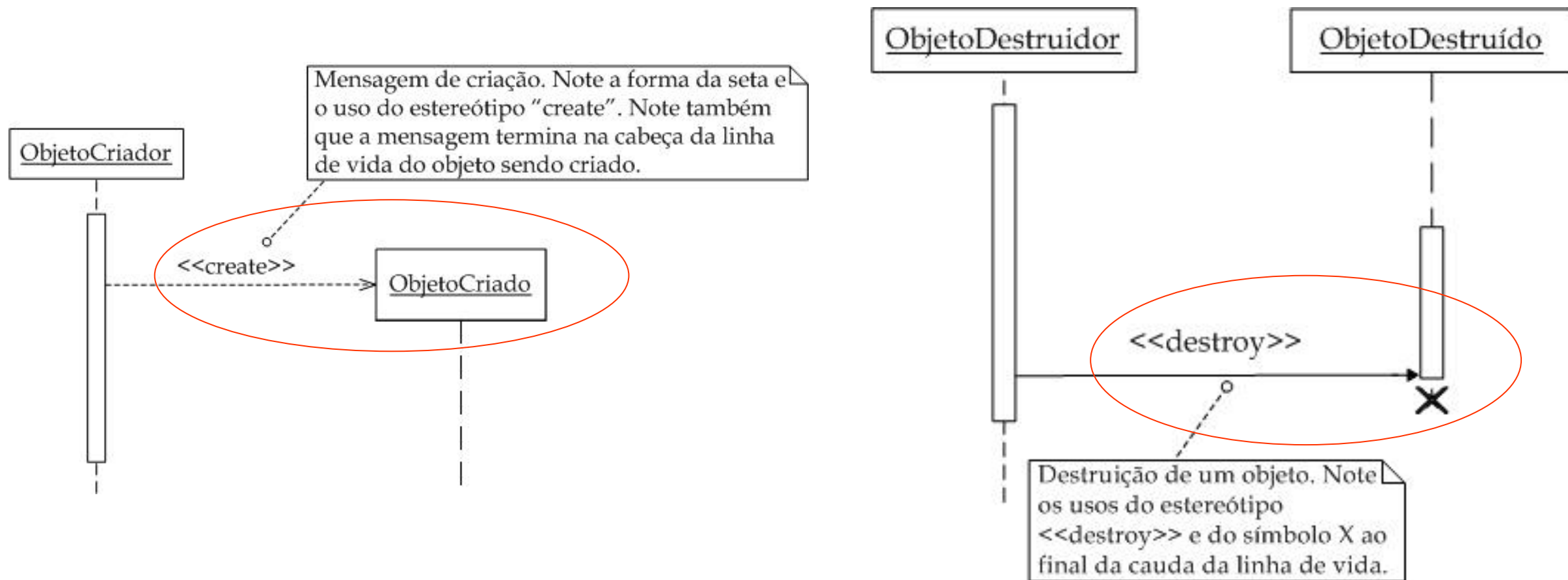


Mensagens reflexivas em um DS

- Em uma mensagem reflexiva (ou auto-mensagem) o remetente é também o receptor
 - Corresponde a uma mensagem para this (self)
 - O que isso significa na prática?



Criação/destruição de objetos em um DS



Mensagem

- Conceito básico da interação entre objetos >>> mensagem
- Um sistema OO é uma rede de objetos que trocam mensagens
 - Funcionalidades são realizadas pelos objetos, que só podem interagir através de mensagens
 - Um objeto envia uma mensagem para outro objeto quando o primeiro deseja que o segundo realize alguma tarefa
- O fato de um objeto “precisar de ajuda” indica a necessidade de este enviar mensagens
- Na construção de diagramas de interação, mensagens de um objeto a outro implicam em operações que classes devem ter

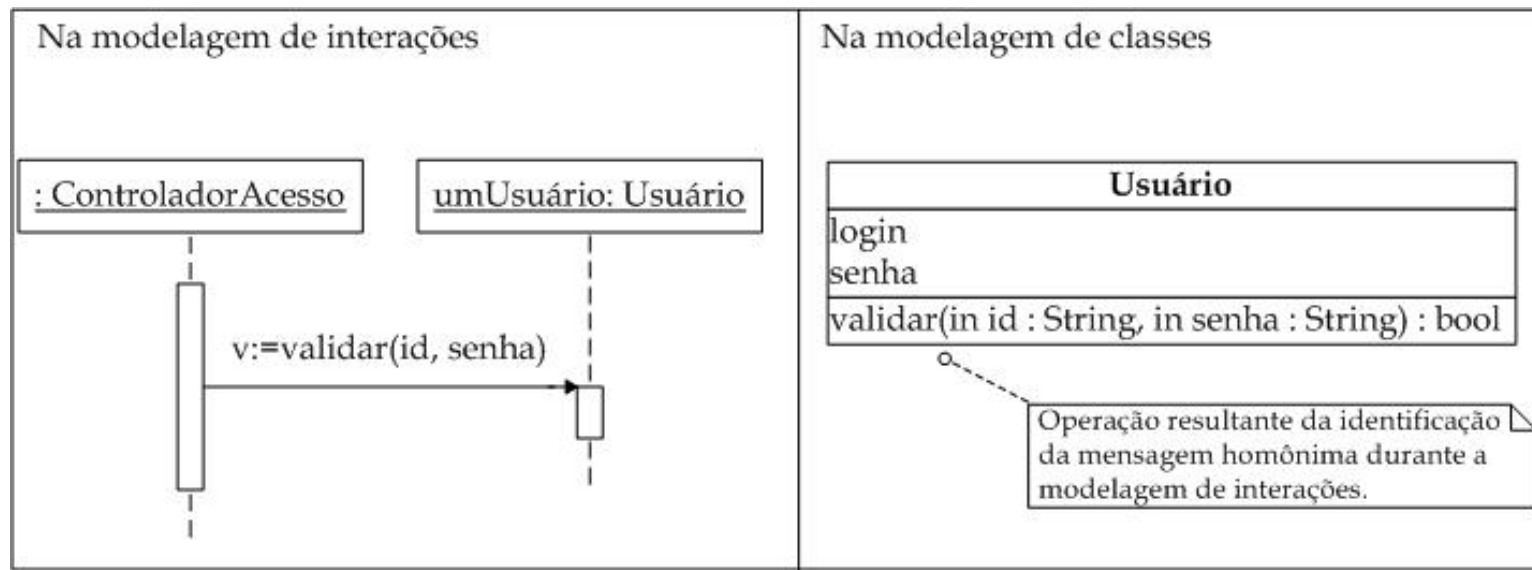
Mensagem

Uma mensagem representa a requisição de um **objeto remetente** a um **objeto receptor** para que este último execute alguma **operação** definida para sua classe

Essa mensagem deve conter **informação** suficiente para que a operação do objeto receptor possa ser executada

Mensagens versus responsabilidades

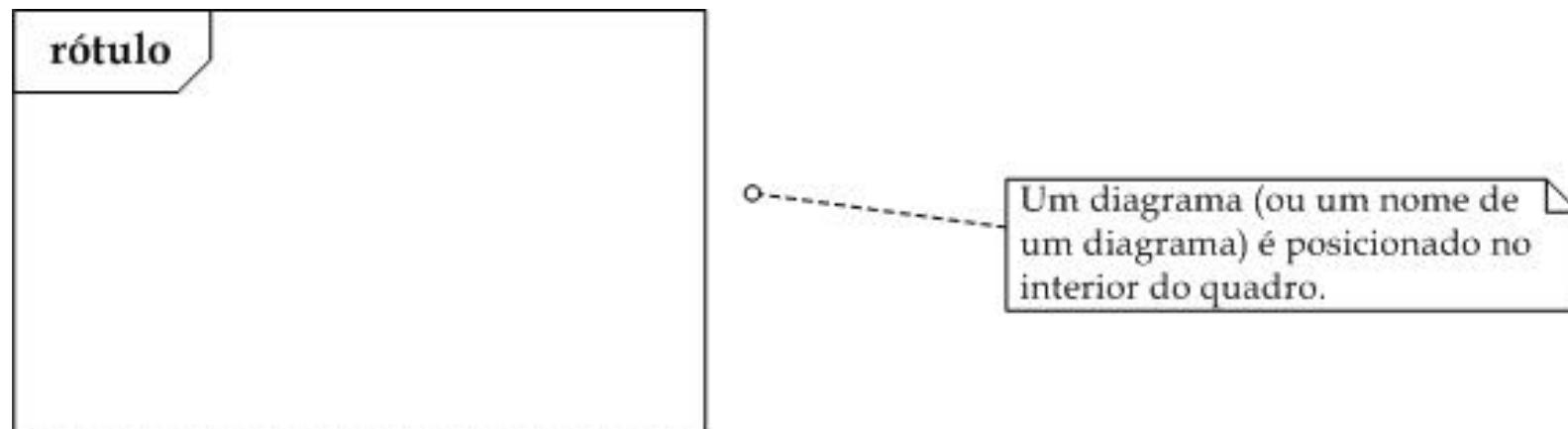
- Qual o objetivo da construção dos diagramas de interação?
 - Identificar mensagens e, em última análise, responsabilidades (operações e atributos)



Uma mensagem implica na existência de uma operação no objeto receptor
A resposta do objeto receptor ao recebimento de uma mensagem é a execução da operação correspondente

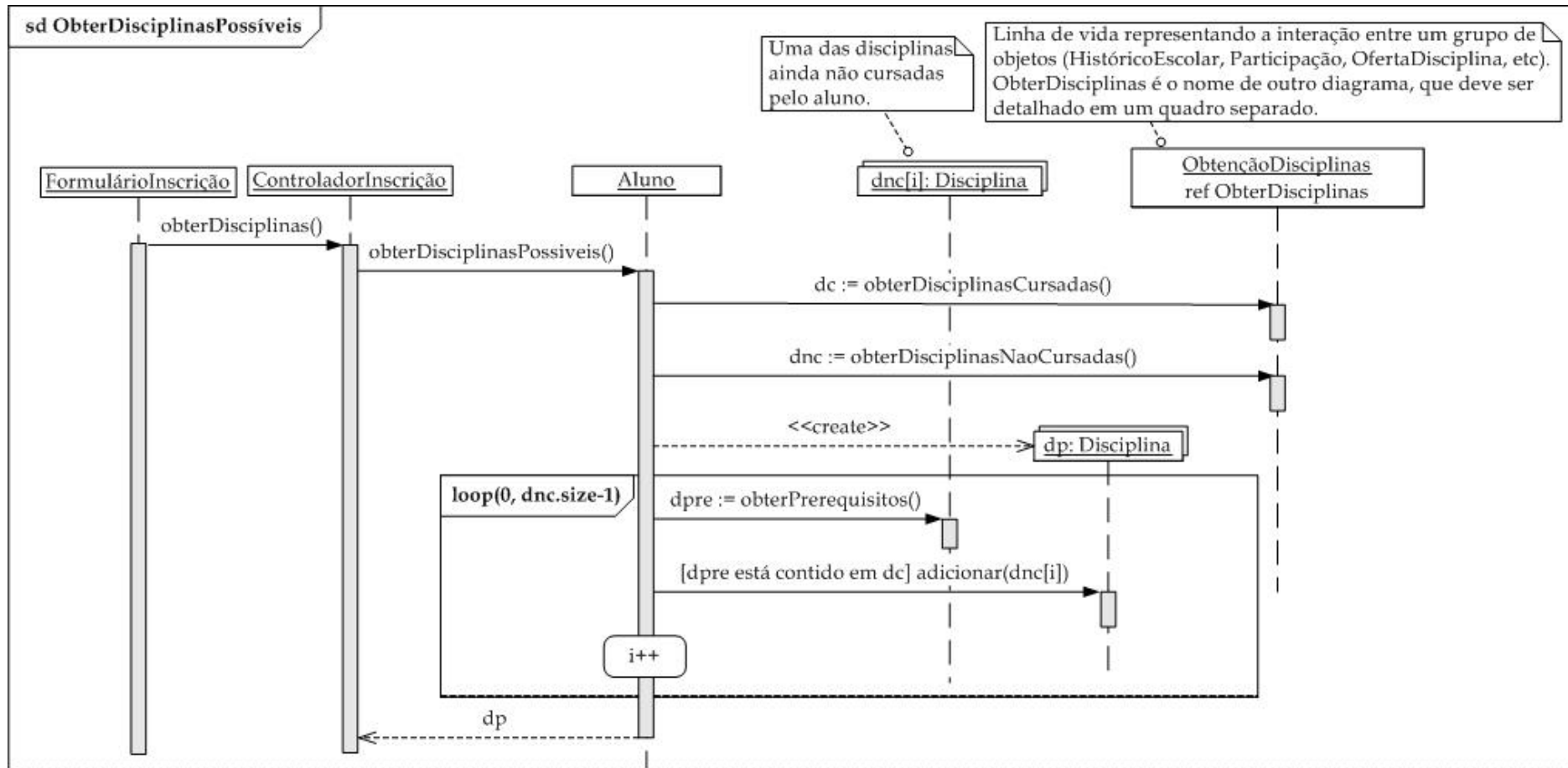
Quadros de interação

- Elemento gráfico, que serve para modularizar a construção de diagramas de sequência (ou de comunicação)
- Objetivos específicos:
 - Dar um nome ao diagrama que aparece dentro do quadro
 - Fazer referência a um diagrama definido separadamente
 - Definir o fluxo de controle da interação
- Notação:



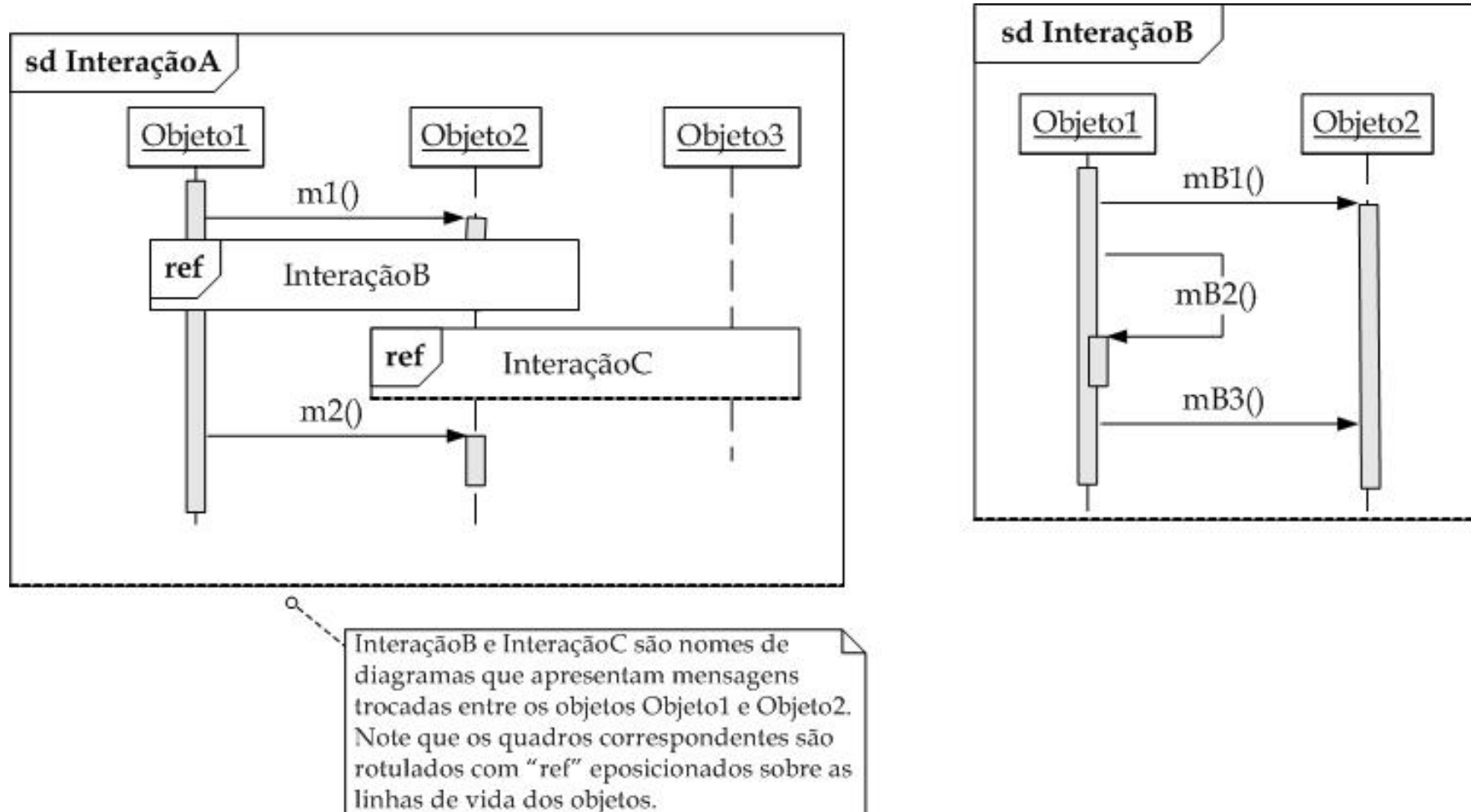
Diagramas nomeados

Dar um nome ao diagrama que aparece dentro do quadro

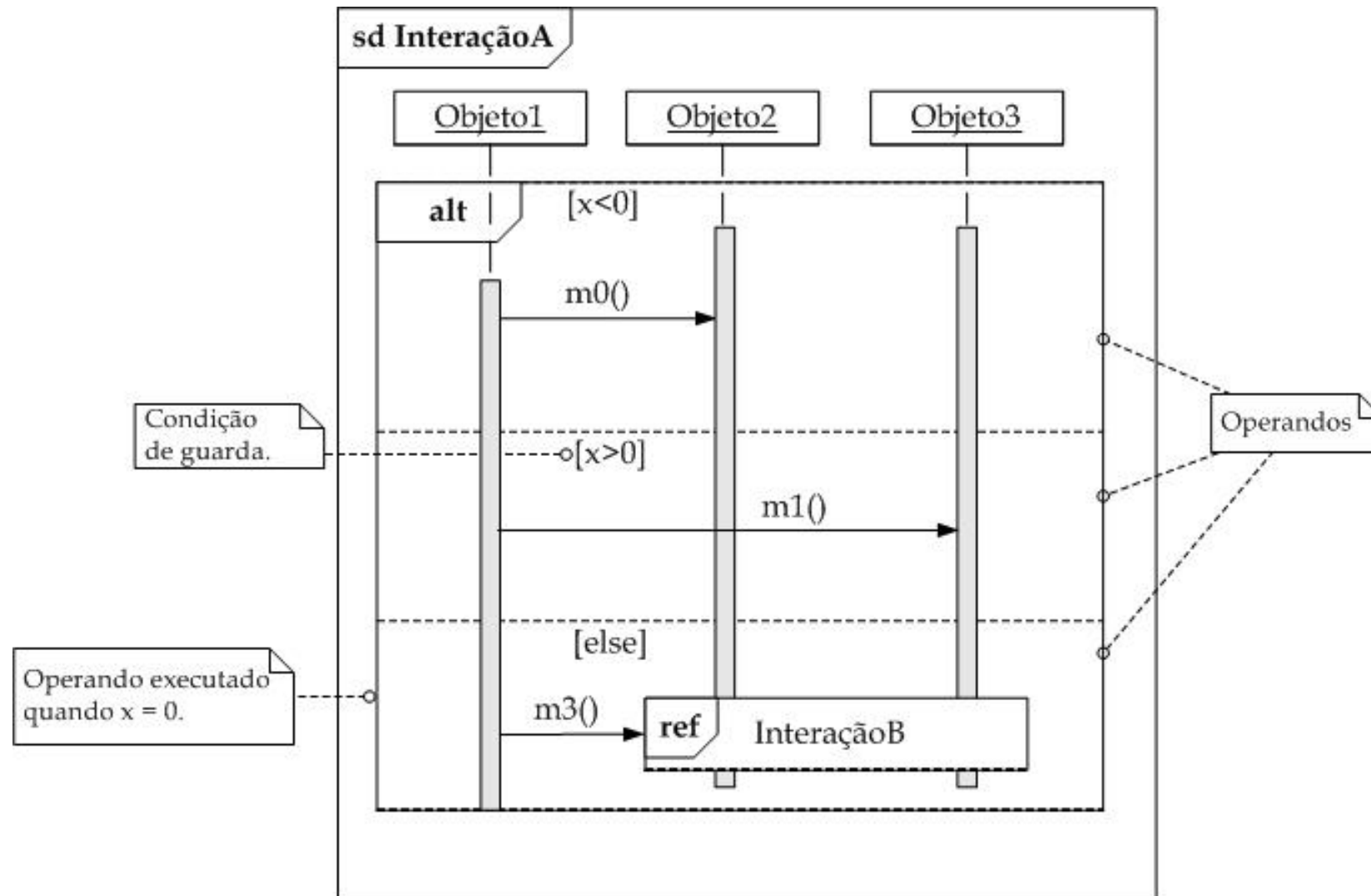


Diagramas referenciados

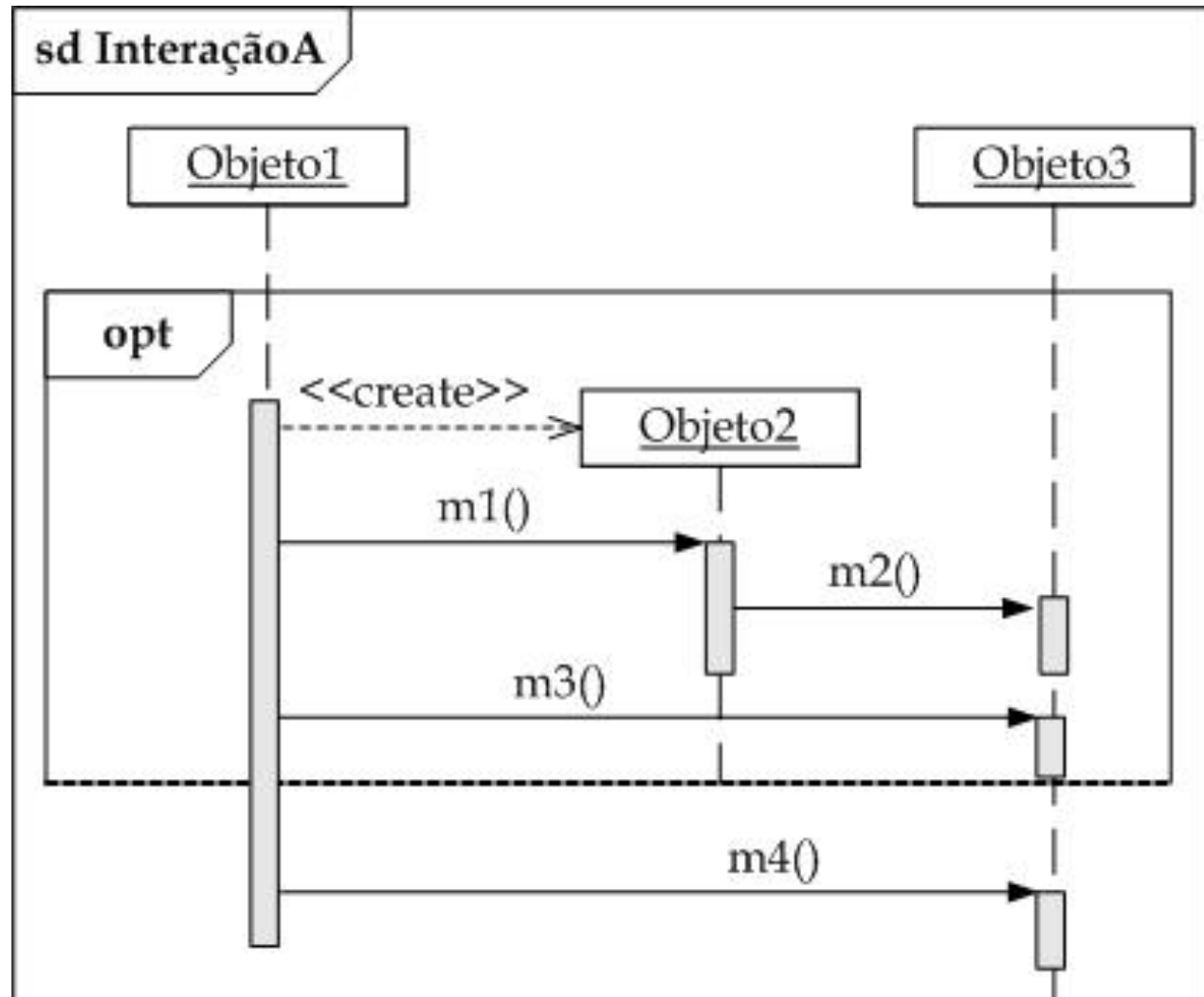
Fazer referência a um diagrama definido separadamente.



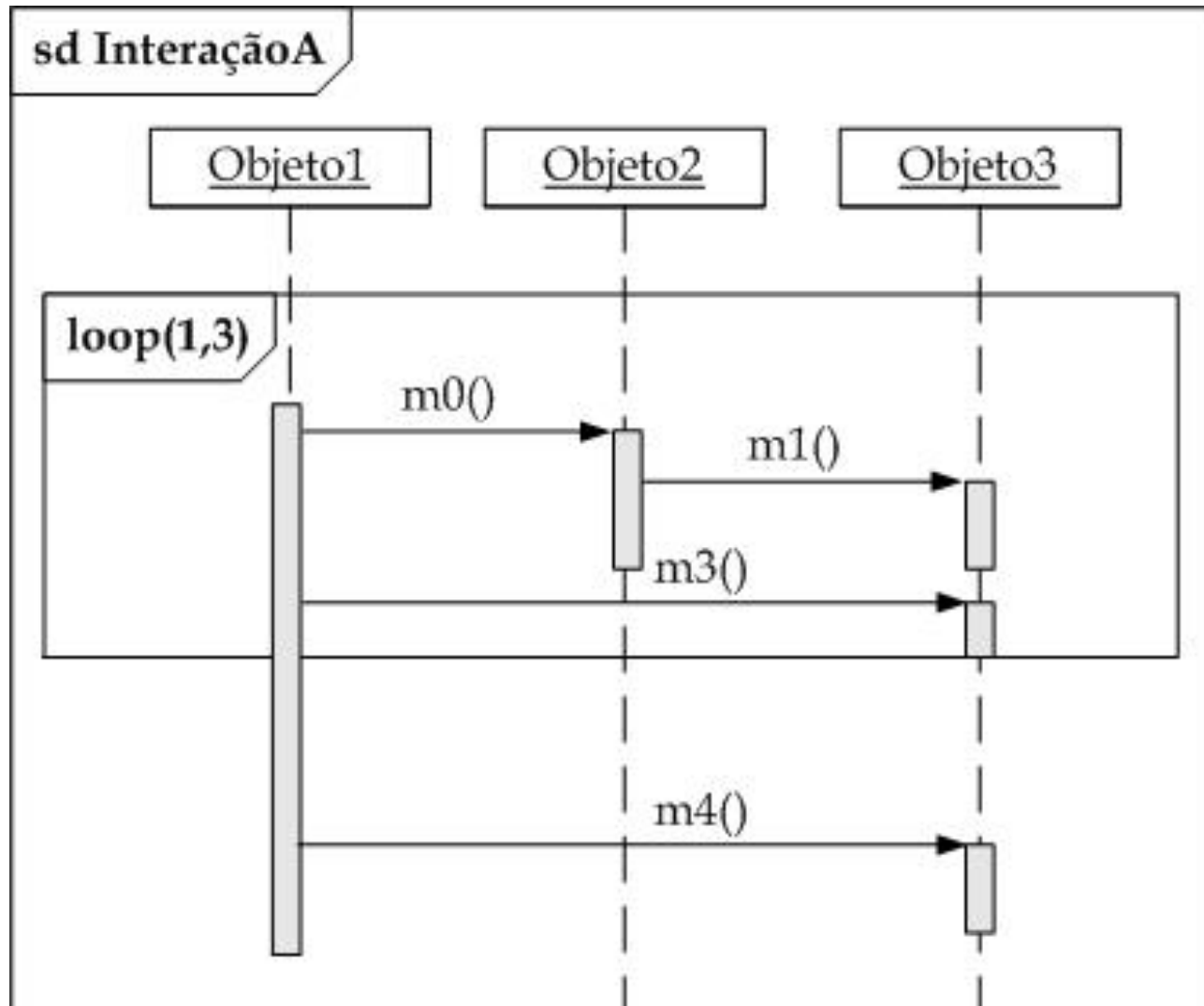
Fluxo de controle: alternativas



Fluxo de controle: opções



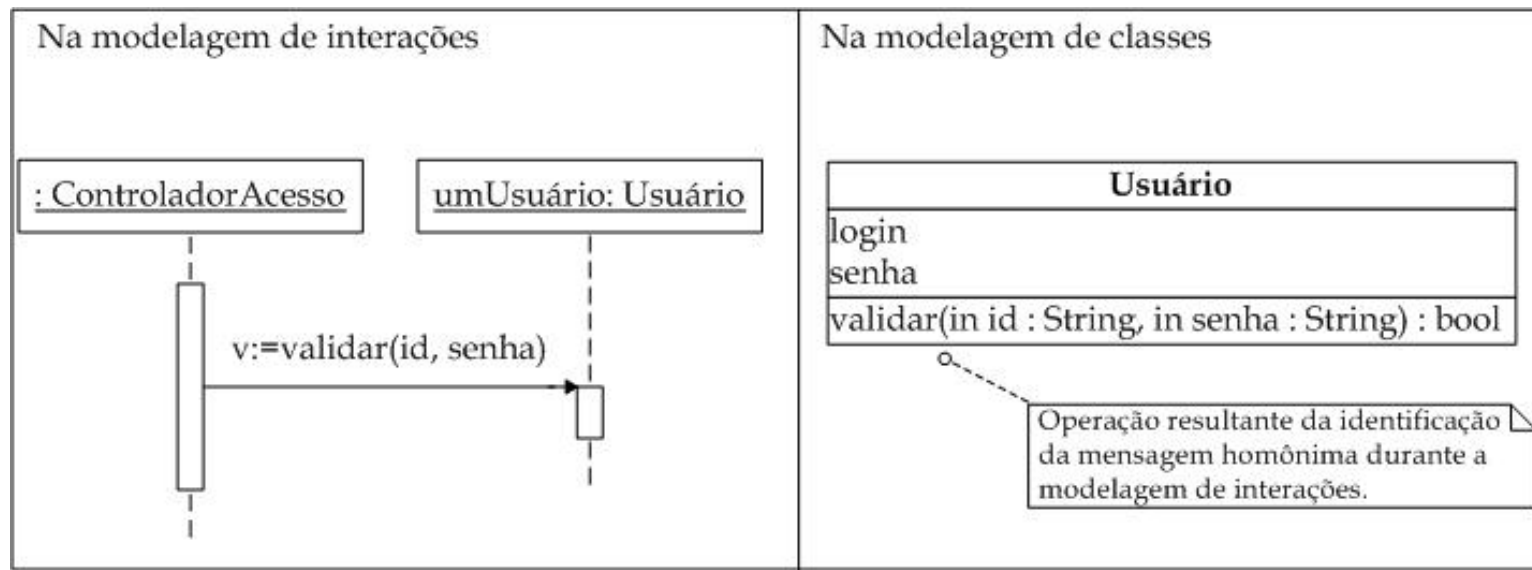
Fluxo de controle: iterações



Construção do modelo de interações

Mensagens versus responsabilidades

- Qual o objetivo da construção dos diagramas de interação?
 - Identificar mensagens e, em última análise, responsabilidades (operações e atributos)



Uma mensagem implica na existência de uma operação no objeto receptor
A resposta do objeto receptor ao recebimento de uma mensagem é a execução da operação correspondente

Alocação de responsabilidades

- Modelagem de interações pode ser um processo cujo objetivo final é **decompor as responsabilidades** do sistema e alocá-las a classes

- Dado um conjunto de N responsabilidades, uma possibilidade é criar uma única classe no sistema para assumir com todas as N responsabilidades
- Outra possibilidade é criar N classes no sistema, a cada um delas sendo atribuída uma das N responsabilidades

- Certamente, as duas alternativas anteriores são absurdas do ponto de vista prático
- Mas, entre as muitas maneiras possíveis de alocar responsabilidades, como podemos saber quais delas são melhores que outras?

Acoplamento e coesão

- Não tem nenhuma receita de bolo!!!
- Para se construir um bom modelo de interações, devemos lançar mão de diversos princípios de projeto
- Dois dos principais princípios >>> acoplamento e coesão

Coesão

- Medida do quão fortemente relacionadas e focalizadas são as responsabilidades de uma classe
- É extremamente importante assegurar que as responsabilidades atribuídas a cada classe sejam altamente relacionadas
- Em outras palavras
 - O projetista deve definir classes de tal forma que cada uma delas tenha alta coesão

Exemplo de Coesão

- Uma classe que representa uma conta corrente deve ser responsável somente por operações relacionadas a conta
- Transferir, sacar, depositar
- Não faz parte de seu objetivo, por exemplo, tratar valores de uma conta poupança

Venda
- item: Item
+ realizarVenda(): void + emitirNotaFiscal(): NotaFiscal + calcularComissao(): float

Figura 12 - Exemplo de classe sem coesão

Acoplamento

- Medida de quão fortemente uma classe está conectada a outras classes, tem conhecimento ou depende das mesmas
- Uma classe com acoplamento fraco (baixo) não depende de muitas outras
 - Uma classe com acoplamento forte é menos inteligível isoladamente e menos reutilizável
- Uma classe com alto acoplamento é mais sensível a mudanças, quando é necessário modificar as classes da qual ela depende

Exemplo de Acoplamento

- Uma conta corrente transfere dinheiro para uma conta poupança
- Uma conta poupança tem seus fundos resgatados para uma conta corrente
- Existe certo nível de acoplamento para que tais funções sejam estabelecidas

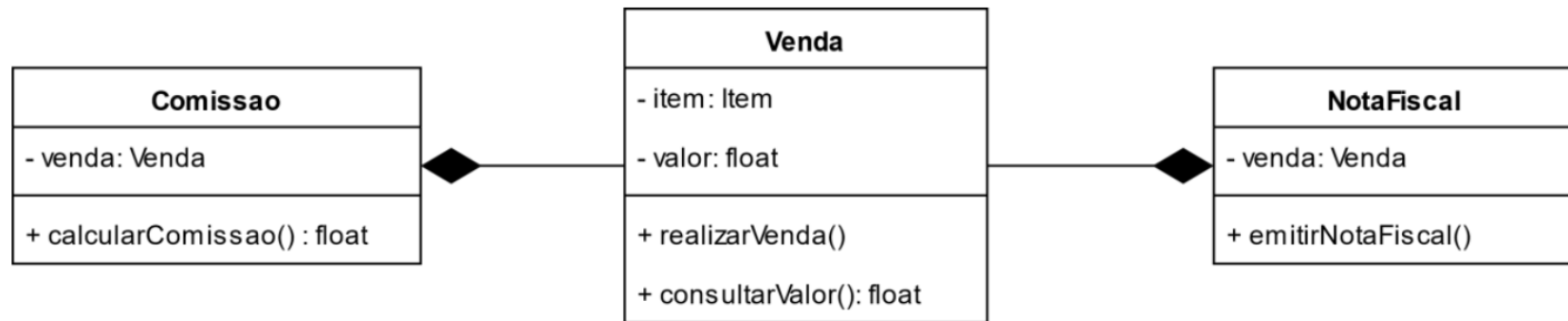


Figura 13 - Modelo de venda com classes coesas

Coesão e Acoplamento

- Conclusão:
 - Criar modelos com alta coesão e baixo acoplamento deve ser um objetivo de qualquer projetista

Dicas para construção do MI

- Identifique classes conceituais que participam em cada caso de uso
 - Estas são as entidades do mundo real que estariam envolvidas na tarefa do caso de uso se este fosse executada manualmente.
 - Ex: Aluno, OfertaDisciplina, Venda, Pagamento, etc
 - Note que classes de fronteira também podem ser classes conceituais.
 - Ex: FormulárioInscrição é um objeto de fronteira (para o caso de uso Realizar Inscrição) que também corresponde a um conceito existente no domínio do problema

Dicas para construção do MI

- Identifique quaisquer classes de software que ajudem a organizar as tarefas a serem executadas
 - Classes que não têm correspondente no mundo real
 - Normalmente são necessárias para manter a coesão das demais classes em um nível alto
 - Ex: classes de fronteira, classes de controle
 - Ex: classes de acesso ao mecanismo de armazenamento, classes de autenticação, etc

Dicas para construção do MI

- Defina também que objetos criam (destroem) outros objetos
 - Na realização de um caso de uso, objetos de entidade podem ser criados pelo objeto de controle, que recebe os dados necessários à instanciação a partir de objetos de fronteira
 - Objetos de entidade também podem ser criados (destruídos) por outros objetos de entidade
 - De uma forma geral, em uma agregação (ou composição), o objeto todo tem prioridade para criar (destruir) suas partes
 - Portanto, em uma agregação (ou composição) entre objetos de entidade, é mais adequado que o objeto todo crie (destrua) suas partes quando requisitado por outros objetos

Dicas para construção do MI

- Verifique a consistência dos diagramas de interação em relação ao **Modelo de Caso de Uso** e ao **Modelo de Classes**
 - Verifique que cada cenário relevante para cada caso de uso foi considerado na modelagem de interações
 - Assegure-se que as mensagens que um objeto recebe estão consistentes com as responsabilidades a ele atribuídas
 - Alguns dos objetos necessários em uma interação já podem ter sido identificados durante a construção do modelo de classes de análise
 - Durante a construção do diagrama de interação, o modelador pode identificar novas classes
 - Atributos, associações e operações também surgem como subproduto da construção dos diagramas de interação

Dicas para construção do MI

- Se certifique de que o objeto de controle realiza apenas a coordenação da realização do caso de uso
 - Como o controlador tem a responsabilidade de coordenação, todas as ações do ator resultam em alguma atividade realizada por esse objeto de controle
 - Isso pode levar ao alto acoplamento; no pior caso, o controlador tem conhecimento de todas as classes participantes do caso de uso
 - Responsabilidades específicas no domínio devem ser atribuídas aos objetos de domínio (entidades)
 - Sempre que for adequado, segundo os princípios de coesão e de acoplamento, devemos fazer com que as classes de domínio enviem mensagens entre si, aliviando o objeto de controle

Dicas para construção do MI

- Faça o máximo para construir diagramas de interação o mais inteligíveis possível
 - Ex: podemos utilizar notas explicativas para esclarecer algumas partes do diagrama de interação que esteja construindo
 - Essas notas podem conter pseudocódigo ou mesmo texto livre
 - Outra estratégia que ajuda a construir um modelo de interações mais inteligível é utilizar os recursos de modularização que a UML 2.0 acrescentou
 - quadros de interação, referências entre diagramas, etc

Dicas para construção do MI

- Utilize o princípio de projeto conhecido como Lei de Demeter
 - Associado ao princípio do acoplamento e impõe restrições acerca de quais são os objetos para os quais devem ser enviadas mensagens na implementação de uma operação:
 - (a) ao próprio objeto da classe (ou self)
 - (b) a um objeto recebido como parâmetro do método
 - (c) a um atributo da classe
 - (d) a um objeto criado dentro do método
 - (e) a um elemento de uma coleção que é atributo da classe
 - A intenção é evitar acoplar excessivamente um objeto e também evitar que ele tenha conhecimento das associações entre outros objetos

Dicas para construção do MI

- Na modelagem de interações, quando definimos uma mensagem, estamos criando uma dependência entre os objetos envolvidos
- Isso é mesmo que dizermos que estamos aumentando o acoplamento entre os objetos em questão
- Portanto, é necessário que o modelador fique atento para apenas definir mensagens que são realmente necessárias
 - Sempre que possível, devemos evitar o envio de mensagens que implique na criação de associações redundantes no modelo de classes
 - Isso porque a adição de uma associação entre duas classes aumenta o acoplamento entre as mesmas

Eventos de sistema

- Correspondem às ações do ator no cenário de determinado caso de uso
- É relativamente fácil identificar eventos de sistemas em uma descrição de caso de uso:
 - Procurar na descrição os eventos que correspondem a ações do ator
- Caso particular em que o ator é um ser humano e existe uma interface gráfica para que o mesmo interaja com o sistema
 - Eventos do sistema são resultantes de ações desse ator sobre essa interface gráfica, que corresponde a objetos de fronteira

Eventos de sistema

- Considere o formulário a seguir, para o caso de uso denominado "Fornecer Grade de Disponibilidades“:

Fornecimento de Grade

Dados do professor

Matrícula: 13091972 Validar Matrícula Nome: Eduardo Bezerra

Disciplinas Disponibilidades

Disciplina: MDSI - Modelagem de Sistemas I +

Código	Nome	Qtd. créditos
MDSI	Modelagem de Sistemas I	04
ENG	Engenharia de Software	04
POO	Programação Orientada a Objetos	04
ADBD	Administração de Bancos de Dados	04

Registrar Grade

Fornecimento de Grade

Dados do professor

Matrícula: 13091972 Validar Matrícula Nome: Eduardo Bezerra

Disciplinas Disponibilidades

Dia Semana: Quarta-feira Hora inicial: 08:30 Hora final: 11:30 +

Dia	Hora Inicial	Hora Final
Segunda-feira	07:00	10:40
Quarta-feira	08:30	11:30
Quarta-feira	14:30	18:00
Sexta-feira	07:00	10:40

Registrar Grade

Eventos de sistema

- Lista de eventos de sistema do formulário:
 - solicitação de validação de matrícula de professor
 - solicitação de adição de uma disciplina à grade
 - solicitação de adição de um item de disponibilidade (dia, hora final e hora final) à grade
 - solicitação de registro da grade
- Importante:
 - Nem todo evento de sistema é originado em um objeto de fronteira correspondente a uma interface gráfica
 - Essa ocorrência pode ser gerada por um ator que não seja um ser humano (ex: outro sistema ou um equipamento)

Procedimento de construção

- Para cada caso de uso, selecione um conjunto de cenários relevantes
 - Incluir o cenário do fluxo principal
 - Considere os fluxos alternativos e de exceção que tenham potencial em demandar responsabilidades de uma ou mais classes
- Para cada cenário selecionado, identifique os eventos de sistema:
 - Posicione o(s) ator(es), objeto de fronteira e objeto de controle no diagrama
 - Para cada passo do cenário selecionado, defina as mensagens a serem enviadas de um objeto a outro
 - Defina as cláusulas de condição e de iteração, se existirem, para as mensagens
 - Adicione multiobjetos e objetos de entidade à medida que a sua participação se faça necessária no cenário selecionado

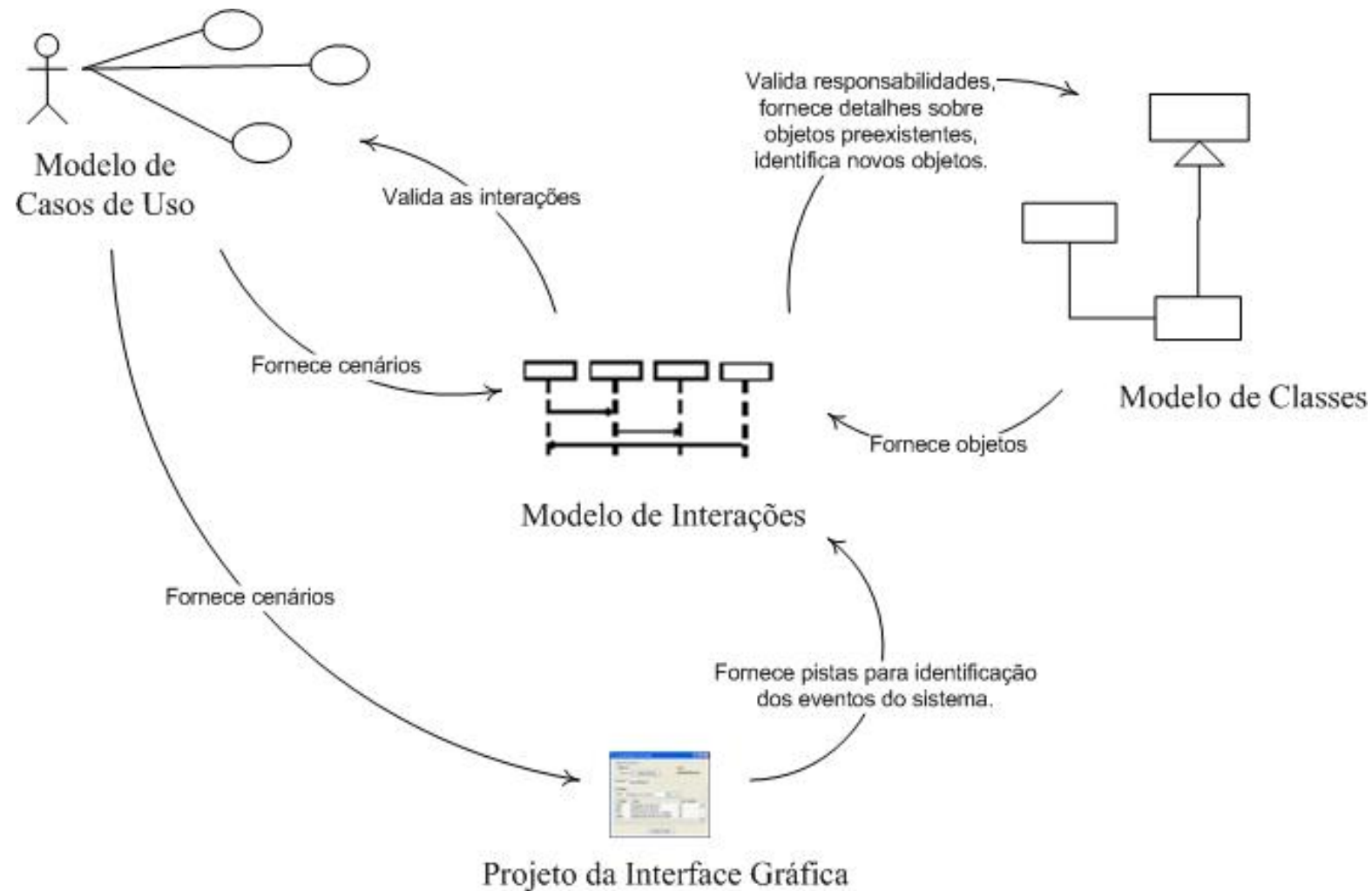
Observações sobre o procedimento

- A definição das mensagens deve ser feita com base nas responsabilidades de cada objeto envolvido:
 - O nome da mensagem
 - Os argumentos de cada mensagem, se existirem
 - O valor de retorno da operação correspondente, se existir
 - Cláusulas de condição e de repetição, se existirem
- A maioria dos objetos já devem ter sido identificados durante a construção do modelo de classes
- Verificar as consistências:
 - Cada cenário relevante para cada caso de uso foi considerado?
 - A mensagens que um objeto recebe estão consistentes com suas responsabilidades?
- As mensagens de um ator a um objeto de fronteira normalmente são rotuladas com a informação fornecida
 - Ex: item de pedido, id e senha, etc

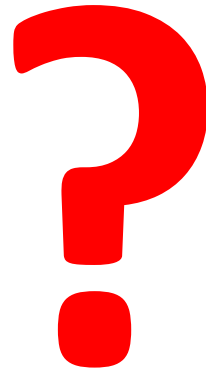
Observações sobre o procedimento

- Mais de um controlador pode ser criado em um mesmo caso de uso, dependendo de sua complexidade
 - O controlador pode mesmo ser suprimido, também em função da complexidade do caso de uso
- Mensagens enviadas pelo objeto de fronteira por conta de um evento de sistema resultam na necessidade de definir operações de sistema no objeto controlador do caso de uso
 - Por exemplo, no do formulário de fornecimento de disponibilidades, o controlador deve possuir as seguintes operações de sistema:
 - validarProfessor(matrícula);
 - adicionarDisciplina(nomeDisciplina)
 - adicionarItemDisponibilidade(dia, horaInicial, horaFinal)
 - registrarGrade()

MI em um processo iterativo



Dúvidas



Exercícios



- Elaborar um diagrama de sequência para:
 - Fluxo básico do caso de uso “Fazer Pedido”
 - Fluxo alternativo 1 do caso de uso “Fazer Pedido”
 - Fluxo alternativo 2 do caso de uso “Fazer Pedido”

Fluxo Básico

- 1) O caso de uso se inicia quando o cliente deseja realizar um pedido;
- 2) O cliente fornece seu nome e endereço;
- 3) O cliente informa os produtos que deseja adquirir;
- 4) O sistema verifica a disponibilidade dos produtos;
- 5) O sistema calcula o valor da compra do cliente;
- 6) O cliente informa a forma de pagamento;
- 7) O cliente confirma a compra;
- 8) O sistema valida as informações;
- 9) O sistema envia uma confirmação da compra via e-mail para o cliente e o caso de uso termina.

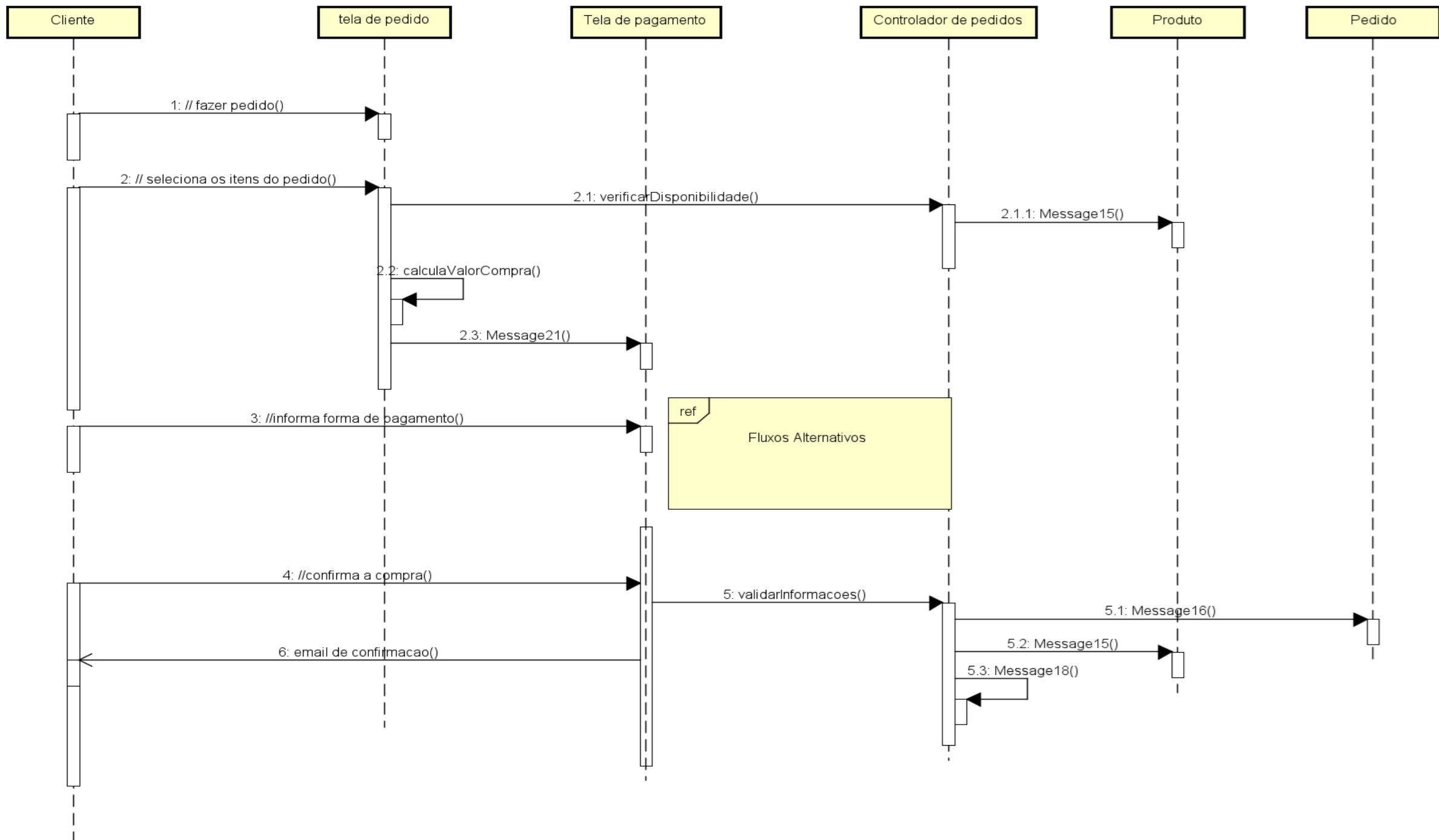


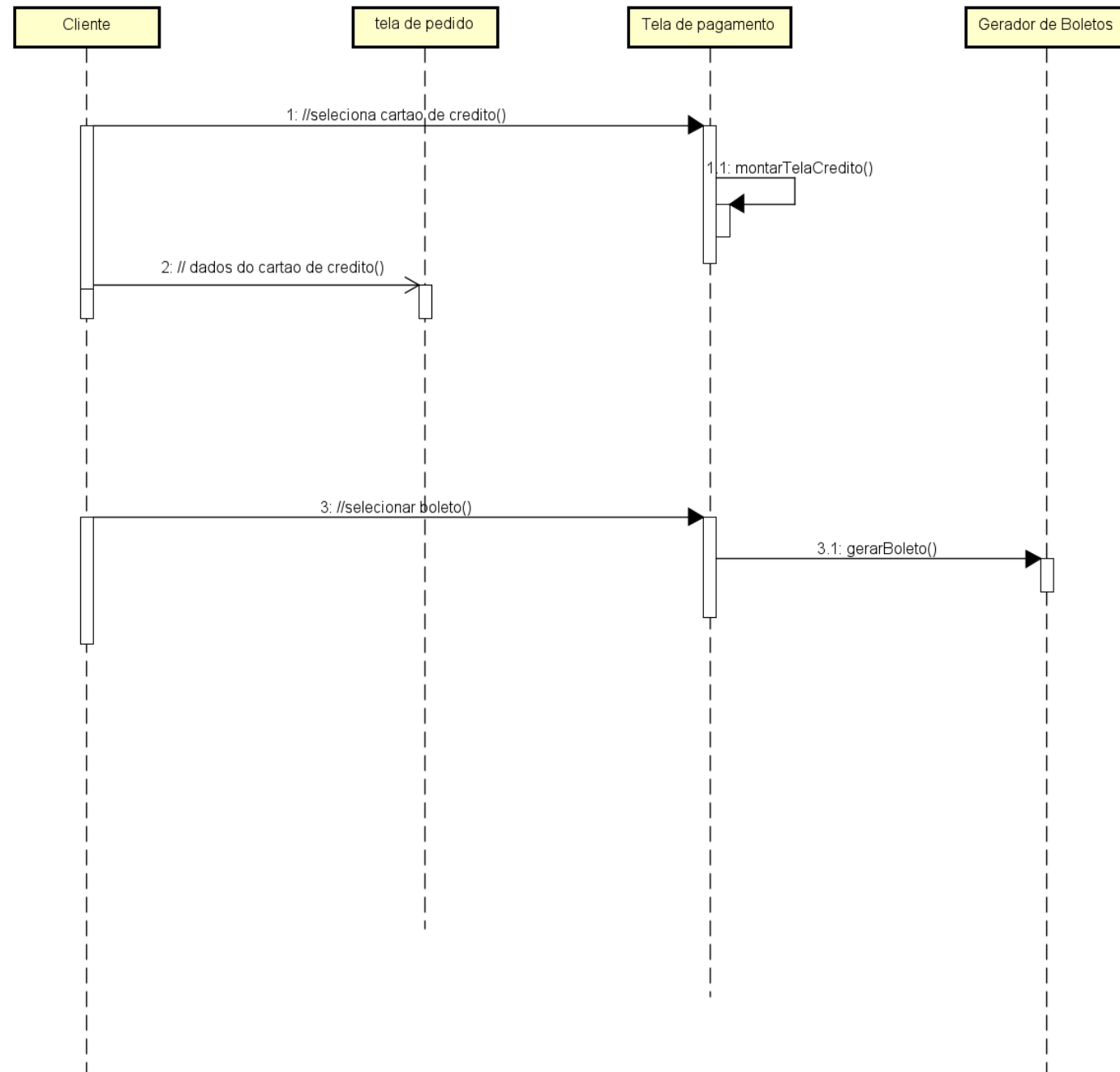
Fluxo Alternativo 1

- 1) Este fluxo se inicia se no **passo 6** do fluxo principal quando o cliente informa a forma de pagamento como sendo através de cartão de crédito
- 2) O sistema solicita o preenchimento do número do cartão e validade;
- 3) O fluxo alternativo retorna ao **passo 7** do fluxo básico.

Fluxo Alternativo 2

- 1) Este fluxo se inicia se no **passo 6** do fluxo principal quando o cliente informa a forma 2) de pagamento como sendo através de boleto bancário;
- 3) O sistema gera um boleto bancário;
- 4) O fluxo alternativo retorna ao **passo 7** do fluxo básico.





Referências Bibliográficas

- Google Imagens
- Freepik
- Análise e Projeto de Sistemas - Universidade Federal do Ceará – UFC
- Campus de Quixadá - Curso de Sistemas de Informação - Prof. Enyo José
- Análise e Projeto de Sistemas – OO – Emanuel Coutinho
- BEZERRA, Eduardo. Princípios de Análise e Projeto de Sistemas com UML. Rio de Janeiro: Elsevier. ISBN-13: 978-85-352-2626-3. 2015.
- Princípios de Análise e Projeto de Sistemas com UML (3a edição) – <https://eic.cefet-rj.br/papsuml3ed/>
- Slides e materiais disponibilizados no site do livro
- RAFAEL TROQUETE - PROGRAMAÇÃO ORIENTADA A OBJETOS: UMA VISÃO CONCEITUAL DOS ELEMENTOS DE MODELAGEM - PONTIFÍCIA UNIVERSIDADE CATÓLICA - ENGENHARIA DE SOFTWARE - Monografia - 2019